CSD 303 → Midsem

A1: a) The equivalence classes of R_L are all the set of strings which are palindromic in nature.

2 eg: 1001, 000111000, etc.

Actually all unique strings will form their own equiv. class

We can conclude the L is not a regular language. A DFA cannot be constructed for checking if a string is a palindrome, because the length of the string is unbounded and would require that we keep

3 track of all the symbols in the string, which is not possible to do with the limited memory (number of states) in a finite automaton.

b) For every string $w_1 \in L_1$, feed it into M_2 and if all of them are accepted, then $\forall w_2 \in L_2$, feed it into M_1. If these are also accepted, we can say with surity that

Can be infinite

$$L(M_1) \subset L(M_2)$$

and

$$L(M_2) \subset L(M_1)$$

How do you check $L_1 \subseteq L_2$?

Hence,

1 $$L(M_1) = L(M_2)$$

$$\Rightarrow M_1 \equiv M_2$$

Since L_1, L_2 can be inf. can't check string by string.

? The decidability property called 'MEMBERSHIP' has a pivotal role to play in this test.

Hence, this question is decidable. ✓

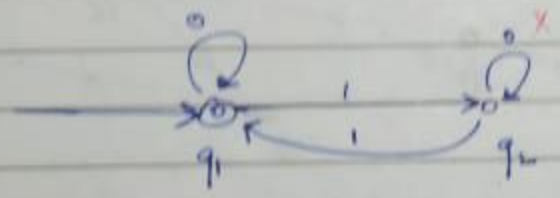
c) Usually, when finite automata have directed cycles in them, it is because we do not know the size of the input and hence wish to generalize the FA for all sizes. In other words, the languages are infinite.

eg: $L = \{ w \mid \text{Last symbol is } 1 \}$

$\Rightarrow L(M) = \{ 1, 01, 11, 001, 011, \ldots \}$

laim: Finite languages can be modelled with DFAs without any directed cycles. However, they may be drawn with them as well

eg: $L = \{ w \mid |w| \leq 4, \text{ Even no. of 1s} \}$



This DFA does not recog just L.

$L(M) = \{ \varepsilon, 0, 00, 000, 0000, 11, 110, 1100 \ldots \}$

// Clearly finite

But,

We see that although all of the above mentioned strings are accepted by the DFA, it also accepts 000000, which is clearly longer than 4 in terms of length. There is no way of monitoring this unless we have separate states for each. why not?

→ By contradiction, we see that M must not have any directed cycles in it Cycle can be in dead state.

The basic reason is that we can 'pump' or repeat the sequence of symbols contained in that cycle to generate an infinite set of strings that are accepted by the DFA

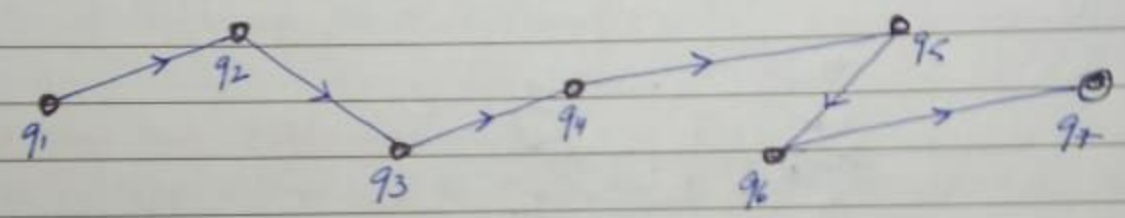
A2: ~~We know that a DFA is constructed with the empty string~~

We know that we start from the initial state in a DFA and the first symbol read takes us to another state.

Thus, for traversing from $q_1 \to q_2 \to \dots \to q_n$, we require exactly $n-1$ symbols.

Incidentally, $n-1$ is also the max no. of edges that can be added to a graph with n vertices without making it cyclic

Consider the example:



We see that no matter how hard we try, we cannot add n transitions in a continuous manner without making it cyclic.

For a string of length n, it should experience a transition at each symbol. After $n-1$ symbols, the transitions in the best case would be such that all states would have been visited once.

By pigeon-hole principle, adding one more transition would forcefully result in a cycle in the graph as there would be an outgoing as well as incoming edge on the same node for the same string.

From the previous answer, it is clear that any directed cycle would result in the language becoming infinite.

Conversely, it follows that if there is no string $s \in \Sigma^{|w| \geq n}$ then an acyclic DFA may be drawn.

In such a DFA, the no of strings that can be accepted is clearly finite as there is a finite number of permutations of the transitions to get to the final state when there is no cycle.

Hence, Proved.

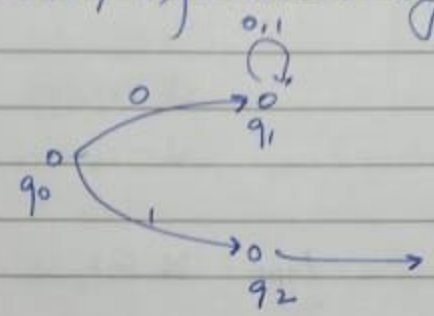
5

iff? Converse?

(5)

A3: a) We know that regular languages can be generated by right linear / left linear regular grammar — (1)

eg:



This is a case of CFG.

$Q_0 \rightarrow 0Q_1$
$Q_1 \rightarrow 0Q_1$
$Q_1 \rightarrow 1Q_1$
$Q_0 \rightarrow 1Q_2$

NFA needed

For every $\delta(q, \sigma) = p$
Create a rule, $Q \rightarrow \sigma P$

5

If p is a final state
Additional rule, $Q \rightarrow \sigma$

Hence, our claim is justified.

Now,
Regular grammars are of the form
$G = (V, \Sigma, P, S)$
Context free grammars are also of the same form

But,
For R.G,
Rules are of the type $V \to w$, $V \to wV'$ } Right linear, $w \in \Sigma^*$, $V' \in V$
OR
$V \to w$, $V \to V'w$ } Left linear

Needs just 2 lns.

For C.F.G,
Rules are of the type $X \to \alpha$, $\alpha \in (V \cup \Sigma)^*$

Clearly, the rules for CFG are far more flexible than those for RG
$\Rightarrow RG \subset CFG$
Hence, every regular grammar is also a context free grammar. —— (II)

We know,
Language generated by CFG
$L = \{w \mid S \xrightarrow{*} w,\ w \in \Sigma^*\} = L(G)$
$\Rightarrow$ If G is a CFG, L is a CFL —— (III)

From (I), (II), (III)
It is evident that every regular language is also a context free language.

6) $L = \{w \mid w \in \Sigma^*$, w has equal no. of 1s and 0s$\}$

$S = 0S1 \mid \varepsilon \Rightarrow S \to 0S1 \;,\; S \to \varepsilon$

i) $S_0 \to S$

ii) ~~X~~ Remove $S \to \varepsilon$

iii) Remove $S_0 \to S$
get, $S_0 \to 0S1$

$S_0 \to S$

$S \to 01$

$S_0 \to 0S1$

iv) $S_0 \to Z_1 1$
$Z_1 \to 0S$

v) $S_0 \to Z_1 W_1$
$W_1 \to 1$
$Z_1 \to W_2 S$
$W_2 \to 0$

Final rules:

$S_0 \to Z_1 W_1$
$W_1 \to 1$
$Z_1 \to W_2 S$
$W_2 \to 0$
$S \to 01$?

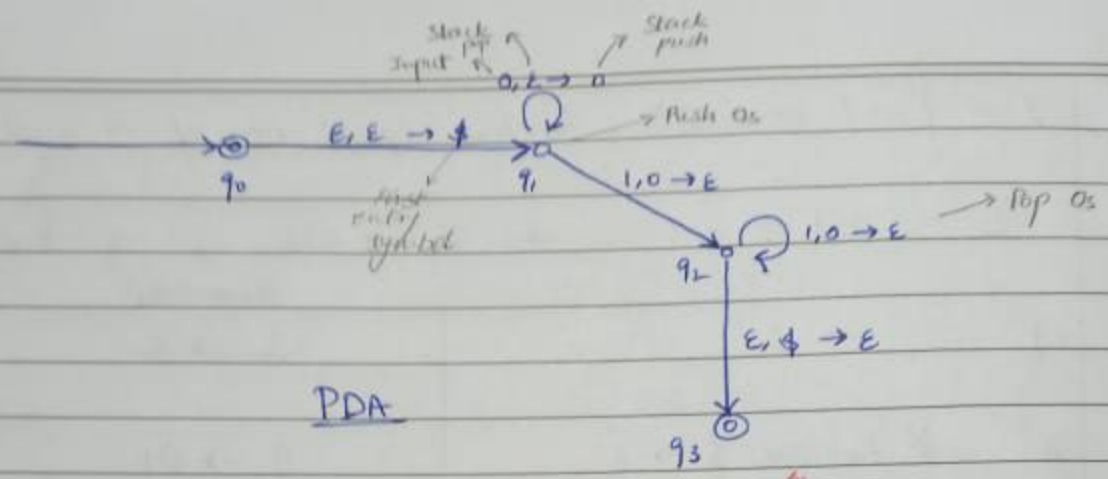
0

ε?

X Will generate finitely many strings.

There is no recursion

c)



PDA

eg : 0011

what about 1100, 1010, 011100 etc?

State	input	Stack
q_0	0011	ε
$q_{0.1}$	0011	$
q_1	011	0$
q_1	11	00$
q_2	1	0$
q_2	ε	$
q_3	ε	ε

✓ ACCEPTED

5